Campus Attendance Platform

A university-wide system: departments, levels, course registration, web check-in and fingerprint verification

Design document | Version 2.0 | 20 October 2026

Decision	Choice
Frontend	React + TypeScript (Vite)
Backend	Go: one REST API, one binary
Hardware firmware	C++ on ESP32 (Arduino framework)
Database	PostgreSQL, with search by course code, title, lecturer, department
Hardware to backend	Same Go API over HTTPS. Scanners never touch the database.
Scope	Multi-faculty, multi-department, all levels. Built multi-institution-ready (every record carries an institution id).

What changed from version 1

- v1 assumed one lecturer, one class and one roster. v2 models the whole university: faculties, departments, levels, academic sessions and semesters, a course catalogue and per-semester course offerings.
- Students **sign up**, are placed in their department and level, **search for classes** and register for them.
- Admins load the official student list so accounts are verified, not self-declared. Roles are scoped (department, faculty, institution).
- Fingerprint hardware is redesigned for many venues, with the real capacity and queue limits spelled out (section 8).
- Section 15 separates what to build in the one-week MVP from what comes later.

1. Purpose and scope

Attendance on paper is easy to fake and slow to consolidate. This platform records attendance for every course in the university in two independent ways and lets lecturers and admins compare them:

- **Web attendance:** the lecturer starts a class session and gets a short code. Students who are in that course enter the code; the app also checks their phone location against the venue.
- **Fingerprint attendance:** a scanner in the venue identifies the student physically present.
- **Comparison and reporting:** per session, per course, per student and per department, including attendance percentage and exam-eligibility lists.

1.1 What a platform for a large university needs

Area	Needed capability
Organisation	Faculties, departments, programmes, levels (100-600), academic sessions and semesters.
Identity	Student sign-up verified against the registry list; lecturer onboarding by invitation; email verification; password reset; optional phone binding.
Courses	Catalogue per department, offerings per semester, co-lecturers, core / elective / general (cross-department) courses.
Discovery	Search and filter classes; register for a course; approval workflow where a department requires it; see what is live now.
Venues and time	Venues with coordinates and radius, class timetable, scanners attached to venues.
Attendance	Code + geolocation, fingerprint scan, late marks, rotating codes, lecturer overrides with audit trail.
Reporting	Attendance percentage, exam-eligibility threshold (commonly 75 percent, configurable), exports, dashboards by department and faculty.
Administration	Scoped roles, CSV imports, device registry, audit log, notifications.
Privacy	Consent for biometrics, minimal data on the server, role-based access, data retention rules.

2. Users and roles

A person has one account and one or more **role assignments**. Each assignment has a scope (the whole institution, one faculty or one department), so a head of department can manage only their own department.

Role	Scope	Can do
Student	Own records	Sign up, search courses, register, check in, view own attendance and percentage, dispute a record.
Lecturer	Courses they teach	Start and end sessions, show code, enrol student fingerprints, approve registrations, view reports, override with a reason.
Course coordinator	One offering	Lecturer rights plus add or remove co-lecturers and set attendance rules for the course.
Department admin (HOD / exam officer)	Department	Manage department courses and offerings, import students, invite lecturers, view department reports.
Faculty admin / Dean's office	Faculty	Same across the faculty's departments; read-only reports by default.
Institution admin (ICT / registry)	Institution	Faculties, departments, academic sessions, venues, devices, role assignments, imports.

Role	Scope	Can do
Super admin (platform owner)	Platform	Create institutions, manage platform settings. Not used day to day.

3. Academic structure and course discovery

3.1 Hierarchy

Level	Examples	Notes
Institution	Obafemi Awolowo University	Tenant. All data is scoped by institution_id.
Faculty	Technology, Science, Arts	
Department	Electronic and Electrical Eng., Computer Science	Students belong to one department; courses belong to an offering department.
Programme	B.Sc. Information and Communication Eng.	Optional in the MVP; needed when a department runs several degrees.
Level	100, 200, 300, 400, 500	Stored on the student; promoted each year by an admin job or CSV import.
Academic session and semester	2026/2027, First Semester	One is marked current. Offerings and registrations hang off the semester.

3.2 Course catalogue vs course offering

- **Course** (catalogue): permanent definition, such as ECE 301, title, units, level, owning department, type (core, elective or general).
- **Course offering:** that course in a particular semester, with its lecturers, registration mode, roster policy and attendance rule. Attendance sessions belong to offerings, not to courses.
- General or service courses (for example GST courses taken by thousands of students from every department) are catalogue entries owned by one department but registered by students from all of them.

3.3 Searching and registering for a class

- A student can search all offerings in the current semester by course code, title, lecturer name, department or level, and filter by department, level and type. Postgres full-text search plus trigram matching (pg_trgm) handles typos such as 'ece301' or 'Cicuits'.
- The student's own department and level are the default filter, so most students see their core courses first, and electives or general courses are one click away.
- **Registration modes** per offering: *auto* (core courses: all students of that department and level are added automatically), *open* (students register themselves), *approval* (lecturer or department admin approves), *closed*.
- **Roster policy** per offering: *registered only* (default) rejects check-ins from unregistered students; *open* accepts them and flags them 'not on roster' so the lecturer decides. Open suits guest lectures and large general courses.
- Home screen for a student: classes live now (from registered courses), next classes today (from the timetable) and attendance percentage per course.

4. Accounts and onboarding

4.1 Student sign-up (verified, not self-declared)

- The institution or department admin imports the official student list as CSV: matric number, full name, department, programme, level, entry year, school email if available. These are **student records**, not accounts yet.

- The student opens Sign up and enters matric number and email. The system finds the record and sends a one-time verification link or code to the school email (or to the email on record). Only then is the account created, bound to that record, with department and level pre-filled and locked.
- If a matric number is not found, the student can submit a request that goes to the department admin for manual approval.
- One record can be claimed once. Department and level can only be changed by an admin (or by the yearly promotion import).

4.2 Lecturers and admins

- Lecturers are **invited** by a department admin (email invitation with a signup link) or sign up with their staff number and wait for approval.
- Admin roles are assigned by the next level up: institution admin assigns faculty and department admins.

4.3 Sign-in security

- Email or matric number plus password, short-lived JWT with refresh token, forced password change on first login for imported accounts.
- **Optional phone binding:** the first device a student uses is remembered; signing in from another device requires admin or lecturer approval. This blocks one phone being used to check in several students, a very common cheat.
- Password reset by email link; rate-limited login; suspended accounts cannot sign in.

5. User journeys

Who	Steps
Student, first week	Sign up with matric number, verify email, confirm department and level. Core courses are auto-registered. Search for electives, register (or request approval). Check that the timetable shows each course.
Student, in class	Open the app, tap the live class, enter the code, allow location, see 'Checked in'. In the venue, place a finger on the scanner. The home screen shows both marks once they are recorded.
Lecturer, before class	Sees assigned offerings and timetable. For fingerprint classes, enrols new students on the scanner from the portal.
Lecturer, in class	Taps Start session (venue and radius come from the timetable). A large code with a countdown appears for the projector, rotating every 60 seconds if rotating codes are enabled. Live counts update.
Lecturer, after class	Ends the session. Opens the comparison: verified, suspected proxy, present without web log, absent. Adds overrides (with reason) for genuine cases such as a student whose phone died.
Department admin	Imports students, invites lecturers, creates offerings for the semester, assigns venues, watches department attendance and the list of students below the exam-eligibility threshold.
Institution admin	Sets up faculties, departments, sessions and semesters, registers venues and scanners, assigns admin roles, runs the yearly promotion import.

6. How attendance is verified

6.1 Session lifecycle

- Lecturer starts a session for an offering. The server copies venue coordinates and radius from the timetable or venue record (adjustable) and generates a code valid for a configurable window (default 10 minutes). Check-ins after a 'late after' time are marked late.
- The student check-in request carries the code, latitude, longitude and accuracy. The server checks: session open, code valid, student registered (or roster policy open), distance within radius, no earlier check-in.
- The scanner sends the matched slot id. The server resolves device plus slot to a student and attaches the scan to the session currently open on that device's venue.
- When the session ends, records are locked. Only overrides, which are audited, can change them.

6.2 Classification shown to lecturers

Web check-in	Fingerprint scan	Status	Meaning
Yes	Yes	**Verified**	Present. Both methods agree.
Yes	No	**Suspected proxy**	Logged in online but never scanned in the room. Code shared or location spoofed.
No	Yes	**Present, no web log**	Scanned but did not log in. Lecturer can accept or flag.
No	No	**Absent**	No evidence of attendance.

For classes with no scanner the report shows web-only statuses (checked in, outside radius, not on roster, absent). Fingerprint is the stronger signal because it needs the person physically at the device.

6.3 Anti-cheating layers

Layer	What it stops	Effort
Time-limited code	Code shared long after class starts	MVP
Geofence with accuracy stored	Check-in from the hostel or the road	MVP
Fingerprint at the venue	A friend checking in for you	MVP
Rotating code on the projector (new code every 30-60 s)	Code forwarded by chat to absent students	Small, recommended
One bound phone per student	One phone used for many check-ins	v2
Roster check	Students from other courses checking in	MVP
Audit log and overrides with reasons	Silent grade or record tampering	MVP

7. Hardware

7.1 Bill of materials per scanner (trimmed for the one-week build)

Part	Qty	Note
ESP32 dev board (30-pin)	1	WiFi and 3 hardware UARTs
R307 fingerprint sensor (AS608 as fallback)	1	R307 preferred for template capacity
OLED 128x64 I2C (SSD1306)	1	Prompts and result text
Active buzzer 3V	1	Driven from a GPIO pin
LED green, LED red	2	With 2 x 330 ohm resistors
Push button	1	Cancel / wake
Breadboard, jumper wires, USB cable, power bank	1 set	Replaces the 18650 pack

Removed from your original list: DS3231 RTC (server provides time), veroboard, capacitors, transistors and flyback diodes (motor or relay parts), the 4.7k and 1k resistors, extra LEDs and buttons, and the 18650 cells.

7.2 Wiring (suggested pins; check your board)

Module pin	ESP32 pin	Note
R307 TX / RX	GPIO16 (RX2) / GPIO17 (TX2)	Cross over: sensor TX goes to ESP32 RX
R307 VCC / GND	3V3 or VIN / GND	Check the rated voltage; R307 often 3.6-6 V, AS608 3.3 V
OLED SDA / SCL	GPIO21 / GPIO22	Address usually 0x3C
Buzzer +	GPIO25	- to GND
Green / Red LED	GPIO26 / GPIO27	Through 330 ohm to GND
Push button	GPIO33	Other leg to GND, INPUT_PULLUP

7.3 Firmware (C++, Arduino framework)

State	Behaviour
Boot	Connect to WiFi, authenticate with the device key, show status.
Idle	Show 'Place finger'. Poll the API every few seconds for the active session on this venue and any pending enrolment.
Match	Sensor searches its templates. Match: green LED, short beep, POST scan (slot id, confidence). No match: red LED, long beep.
Enrol	Triggered by the server. Capture twice, store in the assigned slot, report success.
Offline	Queue scans in memory and send when WiFi returns.

Venue binding: every scanner is registered in the portal and attached to a venue. A session started in that venue is the session the scanner reports to, so lecturers never type session ids on the device. Scanners need WiFi in the venue; a phone hotspot is the backup.

8. Fingerprints at university scale: limits and options

Correction to v1. I said fingerprint templates are hard to export. R307 and AS608 modules can in fact upload and download templates over the serial link, so a central template store is technically possible. I still recommend not storing templates centrally for the MVP (privacy and time), but the option is real, and it is what makes multi-venue use workable. Both options are below.

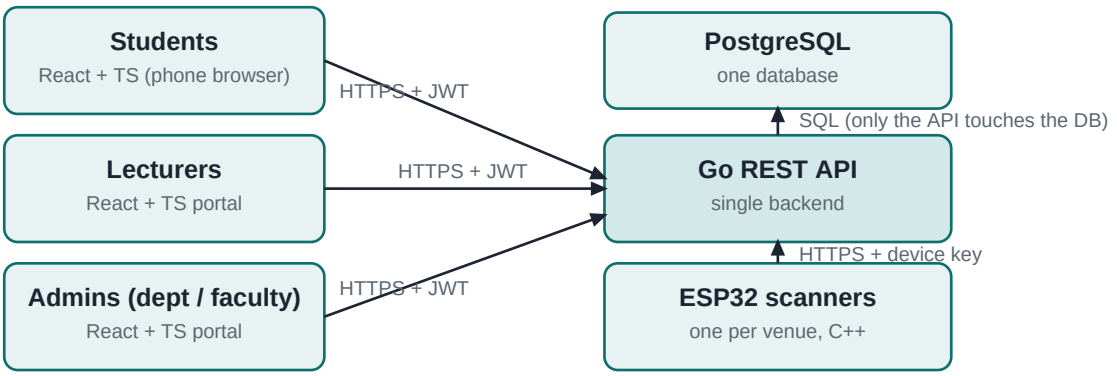
Question	Reality
How many students can one sensor hold?	Module dependent, commonly about 160 (AS608) to about 1000 (R307). Confirm on the datasheet of the exact module.
How fast can one scanner process a class?	Roughly 3-8 seconds per student including queueing. A class of 300 needs 15-40 minutes at one scanner. Large general courses cannot rely on one scanner.
Student attends classes in different venues	Each scanner holds only the templates enrolled on it. The student would need to enrol on every scanner they use (Option A), or have templates loaded per session (Option B).

	Option A: local enrolment (MVP)	Option B: central encrypted templates (v2)
How it works	Template lives only on the sensor where the student enrolled. Server stores slot id and student link.	Server stores encrypted templates. At session start the scanner downloads only the templates of that offering's registered students into the sensor.
Strength	Simple, least biometric data held, fastest to build.	Student enrols once and works at any scanner. Capacity is per class, not per scanner.
Weakness	Re-enrolment per scanner; slot capacity per scanner; does not scale across venues.	More build time. Server now holds biometric data: encryption at rest, access control, consent, retention and breach plan (Nigeria Data Protection Act).
Use for	Pilot: one or two venues, one department.	Faculty- or university-wide rollout.

Practical advice for large classes: use web check-in as the main record and the fingerprint scanner as a *spot check* (for example the first 100 arrivals plus a random sample), or place two or three scanners at the entrance. Say this clearly in the report: the fingerprint column means 'scanned', not 'everyone was required to scan'.

9. Architecture and backend

There is one backend and one database. The web apps and every scanner are clients of the same API, so the rules (who may check in, what counts as present) live in one place.



9.1 Why the scanners use the API and not the database

- Database credentials never sit on a device people can touch.
- The server decides identity, roster membership and timestamps. No RTC is needed.
- Firmware stays small: JSON over HTTPS, no database driver.

9.2 Why Go

Fast builds, one small binary, high concurrency at low memory, simple deployment. When a lecturer announces a code, a few hundred students submit within seconds; Go handles that burst comfortably, though the database is the real limit at that moment, so index `lecture_sessions(code)` and `web_checkins` properly. If the team is not comfortable with Go by day 2, Node + TypeScript keeps the same API contract.

Need	Library
HTTP router	chi (or Gin)
PostgreSQL	pgx v5 with plain SQL or sqlc; pg_trgm extension for search
Migrations	golang-migrate or goose
Auth	golang-jwt, bcrypt (golang.org/x/crypto)
Validation / config	go-playground/validator; environment variables
Rate limiting	httprate or golang.org/x/time/rate
Email (verification, invites)	SMTP or a provider such as Resend or Brevo via their HTTP API
CSV import	encoding/csv from the standard library

9.3 Project structure

```
attendance-api/
  cmd/api/main.go
  internal/
    config/ db/ httpx/            env, pgx pool, JSON/error helpers
    auth/                         login, JWT, signup claim, invites, password reset, middleware
    rbac/                         role + scope checks (institution / faculty / department)
    org/                          institutions, faculties, departments, programmes
    academic/                     academic sessions, semesters
    catalogue/                    courses, offerings, lecturers, search
    registration/                 register, approve, auto-register by dept + level
    venues/                       venues, devices, timetable
    sessions/                     start/end, code generation and rotation
    checkins/                     web check-in, geofence (Haversine), roster rules
    scans/                        device tasks, enrolment, scan intake
    reports/                      classification, percentages, exports
    imports/                      student and staff CSV import, level promotion
    notify/                       email notifications
    audit/                        audit log writer
  migrations/   Dockerfile   Makefile   .env.example
```

9.4 Server rules

- Every query is scoped by institution and checked against the caller's role scope; a department admin can never read another department's data.
- One web check-in and one scan per student per session; repeats are ignored but logged.
- Distance via Haversine; accuracy stored; borderline cases visible to the lecturer.
- All timestamps are UTC from the server, shown in Africa/Lagos time in the UI.
- Every change to attendance after a session ends, and every role change, writes an audit record.

10. Database schema (PostgreSQL)

```
-- organisation
create table institutions (id uuid primary key default gen_random_uuid(), name text not null, short_name
    text, email_domain text);
create table faculties (id uuid primary key default gen_random_uuid(), institution_id uuid not null
    references institutions(id),
  name text not null, code text);
create table departments (id uuid primary key default gen_random_uuid(), faculty_id uuid not null
    references faculties(id),
  name text not null, code text);
create table programmes (id uuid primary key default gen_random_uuid(), department_id uuid not null
    references departments(id),
  name text not null, duration_years int);

-- identity
create table users (
  id uuid primary key default gen_random_uuid(),
  institution_id uuid not null references institutions(id),
  email text not null, password_hash text not null, full_name text not null,
  status text not null default 'pending' check (status in ('pending','active','suspended')),
  email_verified_at timestamptz, must_change_password bool default false,
  bound_device_hash text,                       -- optional phone binding
  created_at timestamptz not null default now(),
  unique (institution_id, email));
create table student_records (                  -- imported from registry, claimed at sign-up
  id uuid primary key default gen_random_uuid(),
  institution_id uuid not null references institutions(id),
  matric_no text not null, full_name text not null, email text,
  department_id uuid not null references departments(id), programme_id uuid references programmes(id),
  level int not null, entry_year int,
  claimed_by uuid unique references users(id),
  unique (institution_id, matric_no));
create table students (user_id uuid primary key references users(id), record_id uuid not null references
    student_records(id),
  department_id uuid not null references departments(id), programme_id uuid references programmes(id),
      level int not null);
create table staff (user_id uuid primary key references users(id), staff_no text, department_id uuid
    references departments(id), title text);
create table role_assignments (
  user_id uuid references users(id),
  role text check (role in ('lecturer','dept_admin','faculty_admin','institution_admin','super_admin')),
  scope_type text check (scope_type in ('department','faculty','institution','platform')), scope_id
      uuid,
  unique nulls not distinct (user_id, role, scope_type, scope_id));   -- needs PostgreSQL 15+

-- academics
create table academic_sessions (id uuid primary key default gen_random_uuid(), institution_id uuid
    references institutions(id),
  name text not null, starts_on date, ends_on date, is_current bool default false);
create table semesters (id uuid primary key default gen_random_uuid(), academic_session_id uuid
    references academic_sessions(id),
  name text not null, starts_on date, ends_on date, is_current bool default false);
create table courses (id uuid primary key default gen_random_uuid(), institution_id uuid references
    institutions(id),
  department_id uuid not null references departments(id), code text not null, title text not null,
  units int, level int, kind text check (kind in ('core','elective','general')), unique (institution_id,
      code));
create table course_offerings (
  id uuid primary key default gen_random_uuid(),
  course_id uuid not null references courses(id), semester_id uuid not null references semesters(id),
  registration_mode text not null default 'open' check (registration_mode in
      ('auto','open','approval','closed')),
  roster_policy text not null default 'registered_only' check (roster_policy in
      ('registered_only','open')),
  min_attendance_pct int not null default 75, unique (course_id, semester_id));
create table offering_lecturers (offering_id uuid references course_offerings(id), lecturer_id uuid
    references users(id),
  is_coordinator bool default false, primary key (offering_id, lecturer_id));
create table course_registrations (
  offering_id uuid references course_offerings(id), student_id uuid references users(id),
  status text not null default 'approved' check (status in ('pending','approved','rejected','dropped')),
  registered_at timestamptz default now(), primary key (offering_id, student_id));
create index on courses using gin (to_tsvector('simple', code || ' ' || title));

-- venues, devices, timetable
create table venues (id uuid primary key default gen_random_uuid(), institution_id uuid references
```

```
    institutions(id),
  name text not null, lat double precision not null, lng double precision not null, radius_m int not
      null default 100);
create table devices (id uuid primary key default gen_random_uuid(), institution_id uuid references
    institutions(id),
  venue_id uuid references venues(id), name text not null, api_key_hash text not null, last_seen_at
      timestamptz);
create table class_schedules (id uuid primary key default gen_random_uuid(), offering_id uuid references
    course_offerings(id),
  venue_id uuid references venues(id), weekday smallint, starts_at time, ends_at time);

-- attendance
create table lecture_sessions (
  id uuid primary key default gen_random_uuid(),
  offering_id uuid not null references course_offerings(id), schedule_id uuid references
      class_schedules(id),
  venue_id uuid references venues(id), started_by uuid references users(id),
  code text not null, code_expires_at timestamptz not null, rotating bool default false, late_after_min
      int default 15,
  venue_lat double precision, venue_lng double precision, radius_m int,
  started_at timestamptz not null default now(), ended_at timestamptz);
create unique index on lecture_sessions(code) where ended_at is null;
create table web_checkins (
  session_id uuid references lecture_sessions(id), student_id uuid references users(id),
  lat double precision, lng double precision, accuracy_m real, distance_m real,
  on_roster bool not null, is_late bool default false, device_hash text,
  created_at timestamptz not null default now(), primary key (session_id, student_id));
create table fingerprint_enrolments (
  id uuid primary key default gen_random_uuid(), student_id uuid references users(id), device_id uuid
      references devices(id),
  slot_id int not null, status text not null default 'pending' check (status in
      ('pending','done','failed')),
  created_at timestamptz default now(), unique (device_id, slot_id), unique (student_id, device_id));
create table fingerprint_scans (
  id bigserial primary key, session_id uuid references lecture_sessions(id), student_id uuid references
      users(id),
  device_id uuid references devices(id), slot_id int not null, confidence int,
  scanned_at timestamptz not null default now(), unique (session_id, student_id));
create table attendance_overrides (
  session_id uuid references lecture_sessions(id), student_id uuid references users(id),
  status text not null, reason text not null, by_user uuid references users(id), at timestamptz default
      now(),
  primary key (session_id, student_id));
create table audit_log (id bigserial primary key, user_id uuid, action text, entity text, entity_id
    text, detail jsonb, at timestamptz default now());
```

The report for a session is the offering's approved registrations left-joined to `web_checkins`, `fingerprint_scans` and `attendance_overrides` on (session, student). Attendance percentage per course is sessions attended divided by sessions held, and the department view lists students below the offering's threshold.

11. API endpoints

Group	Endpoints (examples)	Who
Auth	POST /auth/signup/claim, /auth/verify-email, /auth/login, /auth/refresh, /auth/forgot-password, /auth/reset-password, /auth/accept-invite	Public / invited
Organisation	GET/POST /faculties, /departments, /programmes; GET /departments?faculty_id=	Admins; read for all
Academic calendar	GET/POST /academic-sessions, /semesters; PATCH set current	Institution admin
Catalogue	GET /courses?q=&department;_id=&level;=&kind;=; POST /courses; POST /offerings; PATCH /offerings/{id}	Dept admin; search for all
Discovery	GET /offerings/search?q=&semester;=current&department;_id=&level;=; GET /me/live-classes; GET /me/timetable	Student
Registration	POST /offerings/{id}/register; DELETE same (drop); GET /offerings/{id}/registrations; POST /registrations/{id}/approve	Student / lecturer / dept admin
Imports	POST /imports/students (CSV), /imports/staff, /imports/promote-level	Dept / institution admin
Venues and devices	GET/POST /venues, /devices; POST /devices/{id}/rotate-key; GET/POST /offerings/{id}/schedule	Institution / dept admin
Sessions	POST /offerings/{id}/sessions; POST /sessions/{id}/end; GET /sessions/{id}/code (rotating)	Lecturer
Check-in	POST /checkins (code, lat, lng, accuracy, device hash)	Student
Live and reports	GET /sessions/{id}/live; GET /sessions/{id}/report; GET /offerings/{id}/report; GET /departments/{id}/report; ?format=csv; POST /sessions/{id}/overrides	Lecturer / admins
Fingerprint admin	POST /students/{id}/enrol-fingerprint (device_id)	Lecturer / dept admin
Device	GET /device/tasks; POST /device/scans; POST /device/enrolments/{id}/complete	Scanner (X-Device-Key)
Me	GET /me; GET /me/attendance (per course, with percentage); POST /me/disputes	Student

12. Frontend (React + TypeScript)

Item	Choice
Tooling	Vite, React Router, TanStack Query (polling for live views), Tailwind, React Hook Form + Zod
Structure	One app with role-based routes and navigation, or three route groups: /student, /lecturer, /admin
API types	Shared TypeScript types matching the Go JSON; fetch wrapper that attaches the JWT and refreshes it
Hosting	Vercel or Netlify for the app; Render, Fly.io or Railway for the Go API; managed Postgres

12.1 Screens by role

Role	Screens
Everyone	Sign up (claim record), verify email, login, forgot password, profile.
Student	Home (live classes, next classes, percentage per course); Find classes (search, filters, register); My courses; Check in (code entry, location permission, result); Attendance history per course; Dispute.
Lecturer	My offerings; Start session (big code, countdown, optional rotation, live counts); Roster and registration approvals; Enrol fingerprint; Session report (four statuses, overrides); Course report and CSV.
Department admin	Courses and offerings; Import students (CSV with validation preview); Invite lecturers; Timetable and venues; Department dashboard; Below-threshold list.
Institution admin	Faculties and departments; Academic sessions and semesters; Venues and scanners (register, key rotation, last seen); Role assignments; Level promotion; Audit log.

12.2 Geolocation details

- Use navigator.geolocation.getCurrentPosition with high accuracy; it needs HTTPS and the student's permission.
- Send accuracy with the position. If accuracy is worse than about 100 m, ask the student to retry near a window; the lecturer still sees borderline cases.
- Show clear messages for permission denied, outside radius, wrong or expired code, and not on roster.

13. Reporting and policy

- **Attendance percentage** per student per course = sessions attended / sessions held. Late counts as attended unless the department's rule says otherwise.
- **Exam-eligibility list:** students below the offering's minimum (default 75 percent, configurable per offering or department) are listed for the department admin.
- **Overrides** (medical, device failure, official duty) need a reason and are visible in the audit log. Students can file a dispute; the lecturer resolves it.
- **Notifications** by email: registration approved, session started, falling below the threshold, dispute outcome.
- **Exports:** CSV per session, offering and department; later PDF summaries for the exam office.

14. Security, privacy and limits

- HTTPS everywhere; passwords with bcrypt; short-lived JWTs; role and scope checks on every endpoint; students can read only their own records.
- Scanner keys are stored hashed and can be rotated or revoked. Secrets (database URL, JWT secret) live in environment variables, never in the repo or firmware source.
- Rate limit login, sign-up claim, check-in and device endpoints; log failures.
- Biometrics: written or in-app consent before enrolment, a short privacy notice, retention rules, and a way to delete a student's fingerprint on request. Option A keeps templates off the server entirely; Option B needs encryption at rest, strict access and a breach plan (Nigeria Data Protection Act).
- Student data from the registry is personal data: import over HTTPS only, restrict who can run imports, and keep the import file out of the repository.

Known limits

- Phone GPS is inaccurate indoors and can be spoofed; the fingerprint is the stronger signal, and rotating codes and phone binding add friction.
- Large classes cannot all pass through one scanner; plan spot checks or multiple scanners (section 8).
- A scanner depends on venue WiFi; the offline queue limits loss but cannot replace connectivity.
- Buddy punching at the scanner is only prevented by supervision during enrolment and the lecturer watching the device.

15. What to build, and when

Honest scope check. The full platform above is more than one week for three people. The MVP below proves the idea end to end for one faculty, with real departments, levels, search and both attendance methods. Everything else is designed in so it can be added without redoing the database.

Phase	Includes
MVP (week 1)	Institution, faculties, departments, levels. Student CSV import and verified sign-up (matric + email code). Lecturer accounts created by admin. Course catalogue, offerings, student search and self-registration (open mode, auto-approved). Venues. Sessions with code, expiry and geofence. Optional rotating code. One scanner with local enrolment (Option A), slot-to-student mapping, scan POST. Session report with four statuses, overrides with reason, CSV export, per-course attendance percentage.

Phase	Includes
v2 (weeks 2-4)	Approval registration mode, auto-registration by department and level, timetable and 'live now' list, invitations and role scopes in the UI, below-threshold lists, email notifications, phone binding, disputes, department and faculty dashboards, audit log screens.
v3	Central encrypted templates (Option B), multi-scanner venues, programme-level rules, level-promotion job, PDF summaries, multi-institution onboarding, mobile PWA polish, analytics.

15.1 One-week plan (three people)

Day	Hardware / firmware	Backend (Go)	Frontend (React)
1	Wire and test each module.	Repo, migrations for org + identity, auth, CSV import.	Vite app, routing, login, sign-up claim.
2	Enrol and match on the sensor; WiFi POST.	Catalogue, offerings, search endpoint, registration.	Find classes, register, my courses.
3	Poll /device/tasks, server-driven enrolment.	Venues, sessions, code, check-in + geofence.	Lecturer: start session, code. Student: check in.
4	Scan POST end to end, OLED, LED, buzzer.	Scan intake, enrolment, roster rules.	Live view, enrol-fingerprint screen.
5	Stabilise, tidy wiring.	Reports, percentages, overrides, CSV.	Report page, student attendance view.
6	Real-room test with several fingers.	Rate limits, logging, deploy, seed data.	Admin screens (import, offerings), mobile polish.
7	Demo rehearsal, spare power bank, hotspot.	Freeze, fix.	Final fixes.

15.2 If time runs short, cut in this order

- Rotating codes, email notifications, disputes.
- Admin screens (seed departments and venues with SQL; keep the CSV import).
- OLED student names (show only 'Matched').
- Never cut: verified sign-up, course search and registration, web check-in, fingerprint scan, and the comparison report.

15.3 Demo checklist

- Two departments and two levels seeded, with imported student records and one general course taken by both.
- A new student signs up, finds a course by typing part of its name, registers, and checks in.
- One student who checks in online but is not in the room (Suspected proxy), one who scans only, one who does both.
- Deployed API URL and scanner key tested from the ESP32; hotspot ready as WiFi backup.